



UNIVERSIDAD CENTRAL DE VENEZUELA  
FACULTAD DE CIENCIAS  
ESCUELA DE COMPUTACIÓN  
[6311] Administración de Bases de Datos



## Proyecto #1

# StreamUCV

### *Data Dictionary Interrogator*

Interrogación automatizada del Diccionario de Datos de SQL Server

#### ABD - PROYECTO #1

Administración de Bases de Datos [6311] Semestre: 1-2026  
Profesores: Dra. Concettina Di Vasta / Christian Lechinguero  
Aux. Doc.: Br. Joiner Rojas / Br. Maximiliano B.

#### INTEGRANTES DEL GRUPO

Mateo González — C.I. V-29.900.089  
Samuel Flores — C.I. V-30.416.486

Repositorio GitHub:

[github.com/Mattw-Xproject/ABD-Proy1\\_StreamUCV](https://github.com/Mattw-Xproject/ABD-Proy1_StreamUCV)

Caracas, 21 de Junio de 2026



## Taller #2: Modelos Internos (Arq. SQL SERVER)



### TABLA DE CONTENIDO

|                                                              |    |
|--------------------------------------------------------------|----|
| 1. Descripción General de la Solución .....                  | 4  |
| 1.1 Contexto Institucional y Justificación .....             | 4  |
| 1.2 Objetivo del Proyecto .....                              | 4  |
| 1.3 Alcance y Restricciones Técnicas .....                   | 4  |
| 2. Modelado Conceptual — Diagrama ER Extendido (EER) .....   | 5  |
| 2.1 Introducción al Modelado Conceptual .....                | 5  |
| 2.2 Entidades del Sistema .....                              | 5  |
| 2.3 Relaciones y Cardinalidades .....                        | 5  |
| 2.4 Generalización: Contenido Audiovisual .....              | 6  |
| 2.5 Diagrama EER — Diagrama as Code (Mermaid) .....          | 6  |
| 2.6 Código Fuente del Diagrama (DaC — Mermaid) .....         | 7  |
| 3. Arquitectura de la Aplicación .....                       | 8  |
| 3.1 Estructura de Archivos del Repositorio .....             | 8  |
| 3.2 Descripción Detallada de Módulos .....                   | 8  |
| 3.2.1 config.py — Configuración Centralizada .....           | 8  |
| 3.2.2 db.py — Capa de Acceso a Datos (Gateway) .....         | 8  |
| 3.2.3 queries.py — Repositorio de Consultas T-SQL .....      | 9  |
| 3.2.4 streamucv_app.py — Interfaz Gráfica (831 líneas) ..... | 9  |
| 3.3 Patrones de Diseño Aplicados .....                       | 9  |
| 4. Configuración del Entorno y Ejecución .....               | 11 |
| 4.1 Requisitos del Entorno .....                             | 11 |
| 4.2 Preparación de la Base de Datos .....                    | 11 |
| 4.3 Instalación y Ejecución de la Aplicación .....           | 11 |
| 5. Uso del Diccionario de Datos de SQL Server .....          | 12 |
| 5.1 Concepto de Diccionario de Datos (D/D) .....             | 12 |
| 5.2 Vistas del Catálogo sys.* Utilizadas .....               | 12 |
| 5.3 Mapeo Requerimiento → Vista del D/D .....                | 12 |
| 6. Requerimientos Funcionales Implementados .....            | 14 |
| 6.1 Req 1 & 2 — Tablas e Índices del Esquema .....           | 14 |
| 6.2 Req 3 — Restricciones del Esquema .....                  | 14 |
| 6.3 Req 4 — Detalle de Índices .....                         | 14 |
| 6.4 Req 5 — Triggers .....                                   | 14 |
| 6.5 Req 6 — Tamaño Ocupado por Tabla .....                   | 14 |
| 6.6 Req 7 & 8 — Tamaño de Columnas y Registros .....         | 14 |
| 6.7 Req 9 — Factor de Bloqueo .....                          | 15 |
| 6.8 Req 10 — Estimador de Costos de Consulta .....           | 15 |



## [6311] Administración de Bases de Datos – Proy#1

ABD\_Semestre\_I-2026 Aux. Doc.: Br. Joiner Rojas y Br. Maximiliano B.

Profesores: Dra. Concettina Di Vasta y MSc. Christian Lechinguero



### Taller #2: Modelos Internos (Arq. SQL SERVER)

StreamUCV — Data Dictionary Interrogator

ABD | UCV 2026 - 01

|                                                        |    |
|--------------------------------------------------------|----|
| 7. Cálculos (tr, fb, idx) — Supuestos y Fórmulas ..... | 17 |
| 7.1 Tamaño Estimado del Registro (tr) .....            | 17 |
| 7.2 Factor de Bloqueo (fb) .....                       | 17 |
| 7.3 Estimador de Costos (Req 10).....                  | 17 |
| 8. Repositorio de Código — GitHub .....                | 19 |
| 8.1 Información del Repositorio .....                  | 19 |
| 8.2 Instrucciones de Clonado y Ejecución .....         | 19 |
| 8.3 Resolución de Problemas Comunes.....               | 19 |

## 1. Descripción General de la Solución

### 1.1 Contexto del Proyecto, asunciones y Justificación

StreamUCV es una empresa de medios y entretenimiento que, tras una migración tecnológica completada en 2024, centralizó toda su información audiovisual —series, películas, cadenas, artistas y acuerdos de distribución— en una única instancia de Microsoft SQL Server. Sin embargo, el departamento de Análisis de Datos detectó un problema crítico: los analistas no tenían visibilidad estructurada sobre la propia base de datos. Preguntas operativas básicas como cuántas tablas existen, qué restricciones están activas, qué índices se crearon o cuánto espacio ocupa cada objeto quedaban sin respuesta inmediata.

Ante este contexto, el presente proyecto tiene como objetivo desarrollar una aplicación de escritorio en Python que interroge automáticamente el Diccionario de Datos (D/D) de SQL Server y genere reportes técnicos estructurados del estado del sistema, eliminando la necesidad de codificar consultas manuales complejas para el cuerpo analítico y administrativo.

### 1.2 Objetivo del Proyecto

Desarrollar una aplicación de escritorio nativa en Python 3.10+ que, mediante la biblioteca pyodbc, consulte de forma exclusivamente lectora las vistas de catálogo del sistema sys.\* y de gestión dinámica (sys.dm\_db\_partition\_stats) de SQL Server, y presente los resultados a través de una interfaz gráfica tkinter sin que el usuario deba escribir ninguna consulta SQL manual.

### 1.3 Alcance y Restricciones Técnicas

- La aplicación opera en modo solo lectura: ninguna consulta DML (INSERT, UPDATE, DELETE) ni DDL se emite sobre la base de datos.
- La estructura de la BD StreamUCV —tablas, columnas, restricciones, índices, triggers, datos— no es modificada, eliminada ni ampliada.
- Toda la lógica analítica (cálculo de tr, fb, costos de acceso) reside en la capa Python; no se crean procedimientos almacenados, vistas auxiliares ni objetos en la BD.
- Los parámetros de conexión se centralizan en un único archivo (config.py) para facilitar la adaptación a cualquier instancia local durante la defensa.
- La aplicación genera 10 reportes correspondientes a los requerimientos funcionales definidos en el enunciado del proyecto.

2. Modelado Conceptual — Diagrama ER Extendido (EER)

2.1 Introducción al Modelado Conceptual

Como fase previa al modelado relacional, se construyó un Diagrama Entidad-Relación Extendido (EER) que captura la semántica del dominio de negocio de StreamUCV. El modelo adopta la notación extendida (EER) para representar, además de entidades y relaciones estándar: generalización/especialización con restricciones (Total, Disjunta), relaciones ternarias donde una misma entidad participa en múltiples roles, y atributos con dominio enumerado provenientes de restricciones CHECK definidas en el esquema SQL Server.

El diagrama fue generado como Diagrama as Code (DaC) utilizando el lenguaje Mermaid (flowchart TD), almacenado en el repositorio bajo el archivo streamucv\_eer.mmd. Esto garantiza que el modelo conceptual sea versionable, reproducible y mantenible junto al código fuente.

2.2 Entidades del Sistema

| Entidad               | Tipo                   | Clave Primaria             | Descripción                                                     |
|-----------------------|------------------------|----------------------------|-----------------------------------------------------------------|
| SERIE                 | Fuerte                 | cod_serie                  | Series de TV con tipo (dramática/suspense/cómica) y rating 0–10 |
| PELÍCULA              | Fuerte                 | cod_pelicula               | Películas del catálogo con monto de producción                  |
| CADENA                | Fuerte                 | cod_cadena                 | Cadenas televisivas y de streaming distribuidoras               |
| ARTISTA               | Fuerte                 | cod_artista                | Actores con nombre real, artístico y estado civil               |
| PERSONAJE             | Fuerte                 | cod_personaje              | Personajes ficticios interpretados en series                    |
| HORARIO               | Fuerte                 | cod_horario                | Franjas de transmisión (Todo público / Adultos)                 |
| SEMANA                | PK Compuesta           | (numero_semana, mes, anio) | Semanas de programación con temática y éxito                    |
| CONTENIDO AUDIOVISUAL | Superclase (abstracta) | —                          | Generalización de SERIE y PELÍCULA (ISA Total, Disjunta)        |

2.3 Relaciones y Cardinalidades

| Relación | Tipo     | Participantes                                  | Cardinalidad | Atributos propios                                  |
|----------|----------|------------------------------------------------|--------------|----------------------------------------------------|
| LANZAR   | Binaria  | CADENA — SERIE                                 | N : M        | fecha_lanzamiento (PK), critica, fecha_cancelacion |
| VENTA    | Ternaria | SERIE — CADENA(vendedora) — CADENA(compradora) | N : N : N    | fecha_venta (PK), cambios {si no}                  |

| Relación    | Tipo     | Participantes                                 | Cardinalidad | Atributos propios                                             |
|-------------|----------|-----------------------------------------------|--------------|---------------------------------------------------------------|
| INTERPRETA  | Ternaria | SERIE — PERSONAJE — ARTISTA                   | N : N : N    | fecha_interpretacion (PK), critica, aparicion, cant_episodios |
| PARTICIPA   | Binaria  | ARTISTA — PELÍCULA                            | N : M        | principal, critica, merece_premio                             |
| TRANSMITE   | Ternaria | HORARIO — SERIE(principal) — SERIE(sustituta) | 1 : N : N    | —                                                             |
| INVOLUCRADA | Binaria  | SEMANA — PELÍCULA                             | N : M        | —                                                             |

## 2.4 Generalización: Contenido Audiovisual

SERIE y PELÍCULA comparten atributos comunes (título/nombre, descripción/reseña, cant\_actores) y participan ambas en relaciones con ARTISTA. El modelo EER los agrupa bajo la superclase abstracta CONTENIDO\_AUDIOVISUAL mediante una generalización con restricción Total (todo contenido audiovisual es necesariamente una serie o una película) y Disjunta (ningún elemento puede ser ambas cosas simultáneamente).

CADENA participa en la relación VENTA con doble rol (vendedora y compradora), lo que produce una relación reflexiva ternaria. De manera similar, SERIE participa en TRANSMITE tanto como serie principal como serie sustituta.

## 2.5 Diagrama EER — Diagrama as Code (Mermaid)

El siguiente diagrama fue generado automáticamente a partir del archivo streamucv\_eer.mmd incluido en el repositorio. Representa la fase de modelado conceptual temprano del sistema StreamUCV:

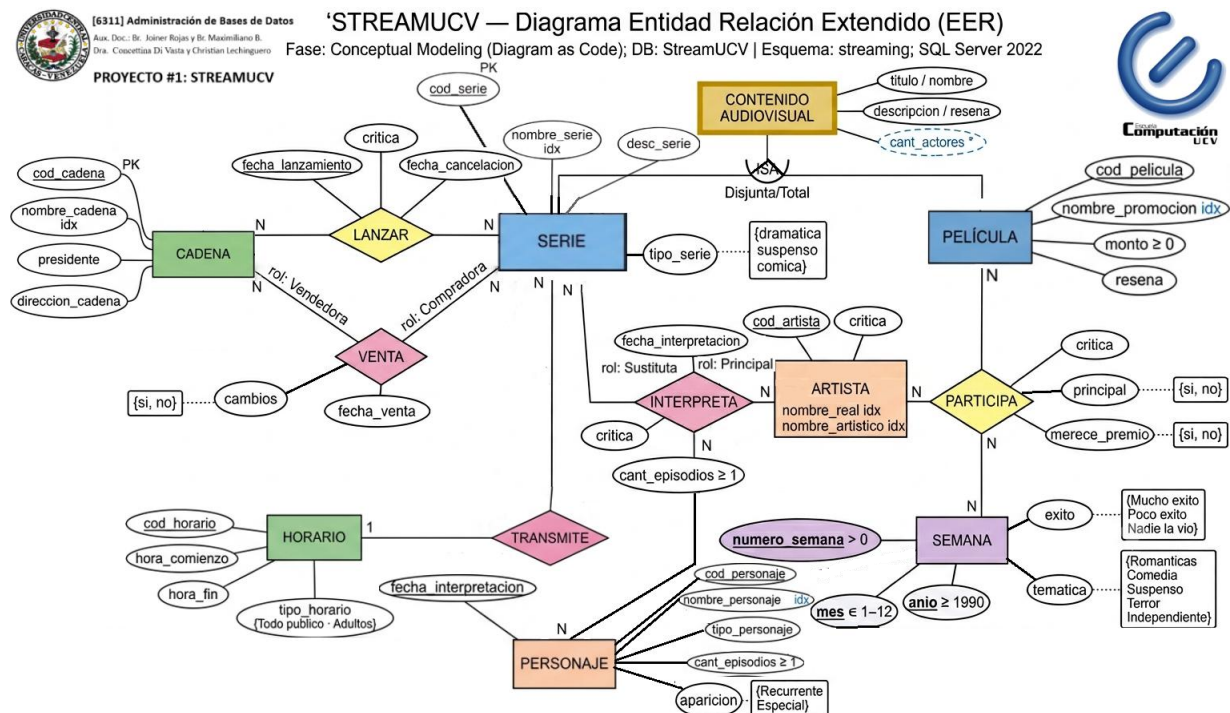


Figura 1. Diagrama Entidad-Relación Extendido (EER) — StreamUCV | Esquema: streaming | SQL Server 2022

## 2.6 Código Fuente del Diagrama (DaC — Mermaid)

El Diagrama As Code(DaC) se almacena en formato Mermaid (.mmd), lo que permite versionarlo en git y regenerarlo en cualquier herramienta compatible (mermaid.live, GitHub, VS Code). El fragmento principal de la estructura es:

```

```.mmd
flowchart TD

%% Generalización: CONTENIDO_AUDIOVISUAL → {SERIE, PELÍCULA}
CONT_AV[(CONTENIDO AUDIOVISUAL)]:::superclase
%% Usamos la notación del triángulo Invertido o trapezoide ([\ /]) para generalización "IS A"
ISA_CONT[\ Total · Disjunta /]:::isa
CONT_AV --> ISA_CONT
ISA_CONT --> SERIE
ISA_CONT --> PELICULA

%% RELACIÓN 1 — LANZAR [Binaria · N : M]
%% Una CADENA lanza N SERIES · Una SERIE es lanzada por N CADENAS
%% PK compuesta: cod_cadena + cod_serie + fecha_lanzamiento
LANZAR{LANZAR}:::relación
CADENA -- "N" --- LANZAR
LANZAR -- "N" --- SERIE

%% RELACIÓN 3 — INTERPRETA [Ternaria · ARTISTA · PERSONAJE · SERIE]
%% Un ARTISTA interpreta un PERSONAJE dentro de una SERIE
%% PK: cod_serie + cod_personaje + cod_artista + fecha_interpretacion
INTERPRETA{INTERPRETA}:::relTernaria
PERSONAJE -- "N" --- INTERPRETA
ARTISTA -- "N" --- INTERPRETA
SERIE -- "N" --- INTERPRETA

%% RELACIÓN 4 — PARTICIPA [Binaria · N : M]
%% Un ARTISTA participa en N PELÍCULAS · Una PELÍCULA tiene N ARTISTAS
%% PK: cod_pelicula + cod_artista
PARTICIPA{PARTICIPA}:::relacion
ARTISTA -- "N" --- PARTICIPA
PARTICIPA -- "N" --- PELICULA

%% RELACIÓN 6 — INVOLUCRADA [Binaria · N : M]
%% PK: numero_semana + mes + anio + cod_pelicula
INVOLUCRADA{INVOLUCRADA}:::relacion
SEMANA -- "N" --- INVOLUCRADA
INVOLUCRADA -- "N" --- PELICULA

%% BLOQUE 7 — ENTIDAD: SEMANA (PK Compuesta de 3 atributos con restricciones de dominio)
SEMANA[(SEMANA)]:::temporal
ase1["<numero_semana> > 0"]:::atributoPK
ase2["<mes> ∈ 1-12"]:::atributoPK
ase3["<anio> ≥ 1990"]:::atributoPK
ase4[exito]:::atributo
ase4_enum[[Mucho exito, Poco exito, Nadie la vio]]:::atributoEnum
ase5[tematica]:::atributo
ase5_enum[[Romanticas, Comedia, Suspenso, Terror, Independiente]]:::atributoEnum

%% =====
%% STREAMUCV – Diagrama Entidad Relación Extendido (EER)
%% Fase: Modelado Conceptual De la BD (Diagrama as Code)
%% BD: StreamUCV | Esquema: streaming | SQL Server 2022
%% Notación: Entidades [(E)] · Relaciones {R} · Atributos ([a])
%% PK subrayada · Derivado(*) · Enumerado{v1, v2} · ISA
%% =====

```

## 3. Arquitectura de la Aplicación

### 3.1 Estructura de Archivos del Repositorio

El proyecto está organizado bajo una arquitectura modular de cuatro capas bien delimitadas. La estructura completa del repositorio publicado en GitHub es la siguiente:

```
ABD-Proy1_StreamUCV/
├── config.py           ← ★ ÚNICO archivo a modificar por entorno
├── db.py              ← Capa de acceso a datos (Gateway)
├── queries.py          ← Repositorio T-SQL (13 consultas al catálogo sys.*)
├── streamucv_app.py    ← Aplicación tkinter completa (831 líneas)
├── streamucv_eer.mmd   ← Diagrama EER en Mermaid (Diagram as Code)
├── requirements.txt    ← Dependencia: pyodbc>=4.0.39
├── README.md           ← Documentación técnica completa
└── sql/
    ├── create_repositorio_sqlserver.sql ← BD StreamUCV + esquema streaming
    ├── create_tables_sqlserver.sql     ← 13 tablas, PK, FK, CHECK, índices
    └── insert_tables_sqlserver.sql      ← Datos de prueba del sistema
```

### 3.2 Descripción Detallada de Módulos

#### 3.2.1 config.py — Configuración Centralizada

Es el único archivo que se modifica al cambiar de entorno (máquina del estudiante a máquina del corrector). Contiene todas las credenciales de conexión SQL Server y los parámetros físicos que son supuestos del proyecto (tamaño de página, tasa de transferencia, altura del árbol B-Tree). Ninguna otra parte del código duplica estos valores.

```
# — config.py — MODIFICAR ESTOS VALORES ANTES DE EJECUTAR —
SERVER      = 'localhost'           # IP o nombre del servidor
DATABASE    = 'StreamUCV'          # No modificar
USERNAME    = 'sa'                  # Login SQL Server
PASSWORD    = 'tu_password'        # Contraseña
DRIVER       = 'ODBC Driver 17 for SQL Server' # Driver ODBC instalado
SCHEMA      = 'streaming'          # Esquema oficial

# Parámetros físicos (supuestos del enunciado)
PAGE_SIZE_BYTES = 8192 # Tamaño de página SQL Server (8 KB)
TRANSFER_RATE_MBS = 17 # Tasa de transferencia en MB/s
BTREE_HEIGHT    = 4    # Accesos estimados: 3 niveles B-Tree + 1 dato
```

#### 3.2.2 db.py — Capa de Acceso a Datos (Gateway)

Implementa el patrón Gateway: abstrae completamente la comunicación de bajo nivel con SQL Server a través de pyodbc. Expone tres funciones públicas que son las únicas que el resto de la aplicación usa para interactuar con la BD. Maneja la apertura y cierre seguro de conexiones y cursores, y captura excepciones de infraestructura de forma que no se propaguen sin contexto al nivel de presentación.

| Función          | Propósito                                                        |
|------------------|------------------------------------------------------------------|
| get_connection() | Retorna una conexión pyodbc activa usando la cadena de config.py |



| Función                    | Propósito                                                              |
|----------------------------|------------------------------------------------------------------------|
| test_connection()          | Verifica la conexión; retorna (True, versión) o (False, mensaje_error) |
| execute_query(sql, params) | Ejecuta un SELECT y retorna (columnas: list[str], filas: list[list])   |

### 3.2.3 queries.py — Repositorio de Consultas T-SQL

Almacena las 13 consultas T-SQL como constantes de módulo nombradas semánticamente (SQL\_REQ1\_2, SQL\_REQ9, SQL\_GET\_TABLES...). Previene el SQL disperso o cableado en los componentes de UI. Todas las consultas son parametrizadas (usan ? como placeholder ODBC) y operan exclusivamente en modo lectura sobre el catálogo sys.\* de SQL Server.

Las consultas más relevantes utilizan:

- CTEs (Common Table Expressions) para separar el cálculo de tamaños de columna de las estadísticas de partición (Req 9).
- STRING\_AGG(...) WITHIN GROUP (ORDER BY ic.key\_ordinal) para reconstruir la lista de columnas de un índice en orden posicional (Req 4) — requiere SQL Server 2022.
- sys.dm\_db\_partition\_stats con filtro index\_id IN (0,1) para obtener row\_count sin duplicar por índice (Reqs 6 y 9).

### 3.2.4 streamucv\_app.py — Interfaz Gráfica (831 líneas)

Implementa la interfaz completa con tkinter puro bajo un patrón MVC ligero. Organiza la UI en cuatro zonas funcionales:

| Zona              | Componente tkinter                      | Función                                                               |
|-------------------|-----------------------------------------|-----------------------------------------------------------------------|
| Barra superior    | tk.Frame + tk.Label                     | Título, nombre de BD/esquema, indicador de conexión (●)               |
| Panel lateral     | tk.Button × 8 + bindings                | Navegación entre los 10 reportes; resaltado activo y hover            |
| Área de contenido | ttk.Treeview + ttk.Scrollbar + tk.Frame | Treeview con filas alternadas, tarjetas de resumen, formulario Req 10 |
| Barra de estado   | tk.Label                                | Mensaje de estado, timestamp de última actualización                  |

## 3.3 Patrones de Diseño Aplicados

| Patrón               | Módulo           | Descripción                                                                  |
|----------------------|------------------|------------------------------------------------------------------------------|
| Configuration Object | config.py        | Una única fuente de verdad para credenciales y parámetros físicos            |
| Gateway              | db.py            | Aísla el acceso a pyodbc del resto de la aplicación                          |
| Query Object         | queries.py       | Consultas SQL nombradas y centralizadas, separadas de la presentación        |
| MVC ligero           | streamucv_app.py | Modelo = db/queries; Vista = Treeview/widgets; Controlador = métodos _show_* |

| Patrón          | Módulo           | Descripción                                                                                              |
|-----------------|------------------|----------------------------------------------------------------------------------------------------------|
| Template Method | streamucv_app.py | <code>_build_report_header()</code> y <code>_build_treeview()</code> reutilizados por todos los reportes |

## 4. Configuración del Entorno y Ejecución

### 4.1 Requisitos del Entorno

| Componente  | Versión mínima           | Notas                                                                                                                |
|-------------|--------------------------|----------------------------------------------------------------------------------------------------------------------|
| Python      | 3.10+                    | tkinter incluido en instalación estándar de Windows                                                                  |
| SQL Server  | 2017+ (recomendado 2022) | STRING_AGG disponible desde 2017; WITHIN GROUP desde 2022                                                            |
| ODBC Driver | 17 for SQL Server        | Descargar desde <a href="https://microsoft.com/en-us/download">microsoft.com/en-us/download</a> si no está instalado |
| pyodbc      | 4.0.39+                  | Única dependencia externa del proyecto                                                                               |
| SO          | Windows 10/11 (o Linux)  | En Linux instalar python3-tk y unixODBC                                                                              |

Para verificar el driver ODBC instalado en Windows: Panel de control → Herramientas administrativas → Orígenes de datos ODBC → pestaña Controladores.

### 4.2 Preparación de la Base de Datos

Ejecutar los tres scripts SQL desde SQL Server Management Studio (SSMS) o sqlcmd, en este orden exacto:

```
-- Paso 1: Crea la BD StreamUCV y el esquema streaming
sqlcmd -S localhost -U sa -P contraseña -i sql/create_repositorio_sqlserver.sql

-- Paso 2: Crea las 13 tablas con restricciones e índices
sqlcmd -S localhost -U sa -P contraseña -d StreamUCV -i sql/create_tables_sqlserver.sql

-- Paso 3: Carga los datos de prueba
sqlcmd -S localhost -U sa -P contraseña -d StreamUCV -i sql/insert_tables_sqlserver.sql
```

### 4.3 Instalación y Ejecución de la Aplicación

```
# 1. Clonar el repositorio
git clone https://github.com/Mattw-Xproject/ABD-Proy1_StreamUCV.git
cd ABD-Proy1_StreamUCV

# 2. Instalar la dependencia Python
pip install -r requirements.txt

# 3. Editar config.py con los datos de la instancia local
#   SERVER / USERNAME / PASSWORD / DRIVER

# 4. Ejecutar la aplicación
python streamucv_app.py
```

## 5. Uso del Diccionario de Datos de SQL Server

### 5.1 Concepto de Diccionario de Datos (D/D)

El Diccionario de Datos (D/D) o Catálogo del Sistema es la base de datos dentro de la base de datos: almacena los metadatos que describen la propia estructura del motor. A diferencia de los datos de negocio —series, actores, cadenas— el D/D almacena la definición de qué tablas existen, de qué tipo son las columnas, cuáles índices aceleran las búsquedas, dónde están almacenados físicamente los datos y cuáles son las estadísticas de ocupación.

En SQL Server, este catálogo se expone a través del esquema sys mediante vistas de catálogo (sys.tables, sys.columns, etc.) y vistas de gestión dinámica (sys.dm\_db\_partition\_stats). La aplicación lo consulta exclusivamente en modo lectura: nunca emite instrucciones DML ni DDL contra la BD StreamUCV.

### 5.2 Vistas del Catálogo sys.\* Utilizadas

| Vista / DMV               | Tipo     | Información que expone                                               |
|---------------------------|----------|----------------------------------------------------------------------|
| sys.schemas               | Catálogo | Esquemas del sistema (filtra por 'streaming')                        |
| sys.tables                | Catálogo | Tablas: nombre, fecha creación/modificación, schema_id               |
| sys.columns               | Catálogo | Columnas: nombre, tipo, max_length, is_nullable, column_id           |
| sys.types                 | Catálogo | Tipos de dato nativos: nombre, user_type_id                          |
| sys.indexes               | Catálogo | Índices: nombre, tipo, is_unique, is_primary_key, fill_factor        |
| sys.index_columns         | Catálogo | Columnas de cada índice: key_ordinal, is_included_column             |
| sys.objects               | Catálogo | Objetos del sistema: PK (PK), FK (F), CHECK (C), UNIQUE (UQ)         |
| sys.triggers              | Catálogo | Triggers DML: nombre, tipo, is_disabled, parent_id                   |
| sys.dm_db_partition_stats | DMV      | Por partición: reserved_page_count, row_count (datos físicos reales) |

### 5.3 Mapeo Requerimiento → Vista del D/D

La siguiente tabla resume cómo cada requerimiento funcional del proyecto mapea a vistas específicas del catálogo:

| Req   | Nombre del Reporte | Vistas sys.* consultadas                    | Qué extrae del D/D                             |
|-------|--------------------|---------------------------------------------|------------------------------------------------|
| 1 & 2 | Tablas e Índices   | sys.tables, sys.indexes, sys.schemas        | Nombre, fechas, cantidad de índices por tabla  |
| 3     | Restricciones      | sys.objects, sys.tables                     | PK, FK, CHECK, UNIQUE con tipo legible         |
| 4     | Detalle de Índices | sys.indexes, sys.index_columns, sys.columns | Columnas clave en orden, unicidad, fill factor |
| 5     | Triggers           | sys.triggers, sys.tables                    | Nombre, tipo, estado, tabla activadora         |

| Req   | Nombre del Reporte   | Vistas sys.* consultadas                                        | Qué extrae del D/D                                   |
|-------|----------------------|-----------------------------------------------------------------|------------------------------------------------------|
| 6     | Tamaño de Tablas     | sys.dm_db_partition_stats                                       | Páginas reservadas → KB → MB, row_count              |
| 7 & 8 | Registros y Columnas | sys.columns, sys.types                                          | max_length por columna → tr por tabla                |
| 9     | Factor de Bloqueo    | sys.columns (CTE),<br>sys.dm_db_partition_stats                 | tr, fb = $\lceil 8192/tr \rceil$ , bloques estimados |
| 10    | Estimador de Costos  | sys.index_columns (key_ordinal=1),<br>sys.dm_db_partition_stats | Costo accesos + tiempo (Index Seek vs Full Scan)     |

## 6. Requerimientos Funcionales Implementados

Cada requerimiento es accesible mediante un botón en el panel lateral izquierdo de la aplicación. Al hacer clic, se ejecuta la consulta correspondiente al catálogo sys.\* y el resultado se presenta en un Treeview con filas alternadas y tarjetas de resumen en la parte superior. No se escribe SQL en ningún momento por parte del usuario.

### 6.1 Req 1 & 2 — Tablas e Índices del Esquema

Lista todas las tablas del esquema streaming con su fecha de creación, fecha de última modificación, la cantidad total de índices definidos (excluyendo el heap, type > 0) y los nombres de esos índices. Las tarjetas de resumen muestran el total de tablas y el total de índices del esquema.

```
SELECT  t.name, t.create_date, t.modify_date,
        COUNT(i.index_id)          AS [N° Índices],
        STRING_AGG(i.name, ' | ') AS [Índices Definidos]
FROM    sys.tables t
JOIN    sys.schemas s ON t.schema_id = s.schema_id
LEFT JOIN sys.indexes i ON t.object_id = i.object_id AND i.type > 0
WHERE   s.name = 'streaming'
GROUP BY t.name, t.create_date, t.modify_date
ORDER BY t.name;
```

### 6.2 Req 3 — Restricciones del Esquema

Muestra todas las restricciones del esquema con nombre, tabla asociada y tipo legible en español. Utiliza sys.objects filtrando type IN ('C','F','PK','UQ') y decodifica el tipo mediante CASE para mostrar 'Clave Primaria (PK)', 'Clave Foránea (FK)', 'CHECK' o 'UNIQUE'.

### 6.3 Req 4 — Detalle de Índices

Para cada índice del esquema lista: tabla, tipo (Clustered / Non-Clustered), si es único, si es PK, fill factor (mostrando 100% cuando el valor es 0, indicando el default del motor) y las columnas clave en orden de posición. Usa STRING\_AGG(...) WITHIN GROUP (ORDER BY ic.key\_ordinal), disponible desde SQL Server 2022.

### 6.4 Req 5 — Triggers

Consulta sys.triggers y sys.tables para listar todos los triggers del esquema. Muestra nombre, tipo (SQL\_TRIGGER), estado (Habilitado / Deshabilitado) según el campo booleano is\_disabled, y la tabla que los activa. Si no existen triggers en el esquema, el Treeview muestra el mensaje 'Sin triggers en el esquema streaming', lo que confirma que la consulta se ejecutó correctamente.

### 6.5 Req 6 — Tamaño Ocupado por Tabla

Accede a sys.dm\_db\_partition\_stats, que acumula contadores de páginas físicas por partición e índice. Agrupa por tabla y suma reserved\_page\_count multiplicado por 8 para obtener el tamaño en KB (cada página SQL Server = 8 KB). También calcula MB y muestra row\_count para el número de registros reales.

### 6.6 Req 7 & 8 — Tamaño de Columnas y Registros

Inspecciona sys.columns unido con sys.types para extraer max\_length en bytes de cada columna. El sistema maneja tres casos especiales:

- Tipos de longitud máxima (-1 para varchar(max) / nvarchar(max)): se estima 8.000 bytes.
- Tipos nvarchar(n): max\_length ya viene en bytes (2 bytes por carácter en UTF-16).
- Tipos numéricos fijos (int=4, decimal(p,s) varía con p): directamente de max\_length.

La aplicación también calcula y muestra en un panel adicional el tamaño estimado del registro (tr) por tabla, que es la suma de todos los max\_length de sus columnas.

### 6.7 Req 9 — Factor de Bloqueo

Implementa las fórmulas requeridas mediante una CTE que separa el cálculo de tamaño de columnas de las estadísticas de partición:

```
WITH col_sizes AS (
    SELECT t.name, t.object_id,
           COUNT(c.column_id) AS num_cols,
           SUM(CASE WHEN c.max_length = -1 THEN 8000
                     ELSE c.max_length END) AS tr_bytes
    FROM sys.columns c JOIN sys.tables t ON c.object_id = t.object_id
    JOIN sys.schemas s ON t.schema_id = s.schema_id
    WHERE s.name = 'streaming'
    GROUP BY t.name, t.object_id
),
part_stats AS (
    SELECT p.object_id,
           SUM(p.row_count) AS rows_total
    FROM sys.dm_db_partition_stats p
    WHERE p.index_id IN (0, 1) -- heap o clustered, sin duplicar
    GROUP BY p.object_id
)
SELECT cs.tabla_name,
       cs.tr_bytes AS [tr (Bytes/Reg)],
       8192 / cs.tr_bytes AS [Factor Bloqueo (fb)],
       CEILING(ps.rows_total * 1.0
               / NULLIF(8192 / cs.tr_bytes, 0)) AS [Bloques Estimados]
FROM col_sizes cs LEFT JOIN part_stats ps ON cs.object_id = ps.object_id;
```

### 6.8 Req 10 — Estimador de Costos de Consulta

Implementa un formulario interactivo donde el analista selecciona una tabla y una columna. La aplicación verifica en sys.index\_columns si la columna es clave líder (key\_ordinal = 1) de algún índice, obtiene el número de páginas reservadas y computa el costo en dos escenarios:

| Escenario                            | Estrategia          | Accesos a disco                     | Cálculo del tiempo                                            |
|--------------------------------------|---------------------|-------------------------------------|---------------------------------------------------------------|
| Columna tiene índice (key_ordinal=1) | Index Seek (B-Tree) | 4 (≈ 3 niveles + 1 página de datos) | $t = (4 \times 8192) / (17 \times 1.048.576)$<br>≈ 0.001843 s |
| Sin índice en la columna             | Full Table Scan     | reserved_page_count de la tabla     | $t = (\text{páginas} \times 8192) / (17 \times 1.048.576)$ s  |

La pantalla muestra el cálculo paso a paso con valores concretos: bytes leídos totales, tasa de transferencia en bytes/segundo y tiempo resultante expresado con 6 decimales para apreciar la diferencia entre ambos escenarios.





## 7. Cálculos, Supuestos y Fórmulas

### 7.1 Tamaño Estimado del Registro (tr)

El tamaño estimado del registro se calcula como la suma de los tamaños máximos de todas sus columnas, según los tipos de datos definidos en la base de datos:

$$tr = \sum \max\_length(columna_i) \quad \text{para } i = 1..N \text{ columnas de la tabla}$$

La tabla siguiente documenta cómo SQL Server reporta max\_length en sys.columns para los tipos usados en StreamUCV:

| Tipo SQL Server | max_length reportado      | Supuesto aplicado                                         |
|-----------------|---------------------------|-----------------------------------------------------------|
| varchar(n)      | n bytes                   | 1 byte por carácter ASCII / Latin1                        |
| nvarchar(n)     | $n \times 2$ bytes        | UTF-16: 2 bytes por carácter (no usado en este esquema)   |
| int             | 4 bytes                   | Tamaño fijo del tipo entero de 32 bits                    |
| decimal(3,1)    | 5 bytes                   | Precisión 1–9 → 5 bytes de almacenamiento                 |
| decimal(18,2)   | 9 bytes                   | Precisión 10–19 → 9 bytes de almacenamiento               |
| date            | 3 bytes                   | Tamaño fijo del tipo fecha SQL Server                     |
| time(0)         | 3 bytes                   | time(0–2) → 3 bytes                                       |
| varchar(max)    | -1 → estimado 8.000 bytes | Longitud variable sin límite: se usa estimado conservador |

Supuesto: se usa el máximo posible por columna (longitud fija), sin considerar overhead de cabecera de fila ni bytes de mapa de nulos. Este enfoque es el indicado por el enunciado del proyecto para uniformidad de corrección.

### 7.2 Factor de Bloqueo (fb)

$$fb = \lfloor 8.192 / tr \rfloor$$

donde:

8.192 bytes = tamaño de página de datos SQL Server (supuesto del proyecto)

tr = tamaño estimado del registro (bytes)

$\lfloor \rfloor$  = función piso – división entera; un registro no se parte entre páginas

El número de bloques estimados para almacenar todos los registros de la tabla es:

$$\text{Bloques estimados} = \lfloor \text{Nº registros} / fb \rfloor$$

### 7.3 Estimador de Costos (Req 10)

Con índice (Index Seek, árbol B-Tree):

```
Accesos      = 4                (≈ 3 niveles de árbol + 1 lectura de página de datos)
Bytes leídos = 4 × 8.192 = 32.768 bytes
Tiempo (s)   = 32.768 / (17 × 1.048.576) = 32.768 / 17.825.792 ≈ 0.001843 s
```

Sin índice (Full Table Scan):

```
Accesos      = reserved_page_count (todas las páginas reservadas de la tabla)
Bytes leídos = páginas × 8.192 bytes
Tiempo (s)   = (páginas × 8.192) / (17 × 1.048.576)

Tasa de transferencia: 17 MB/s = 17 × 1.024 × 1.024 = 17.825.792 bytes/s
```

## 8. Repositorio de Código — GitHub

### 8.1 Información del Repositorio

| Campo                | Valor                                                             |
|----------------------|-------------------------------------------------------------------|
| URL                  | https://github.com/Mattw-Xproject/ABD-Proy1_StreamUCV             |
| Propietario          | Mattw-Xproject (Mateo Daniel González P.)                         |
| Visibilidad          | Público                                                           |
| Rama principal       | main                                                              |
| Lenguaje principal   | Python 3.10+                                                      |
| Archivos principales | config.py, db.py, queries.py, streamucv_app.py, streamucv_eer.mmd |

### 8.2 Instrucciones de Clonado y Ejecución

```
# Clonar el repositorio
git clone https://github.com/Mattw-Xproject/ABD-Proy1_StreamUCV.git
cd ABD-Proy1_StreamUCV

# Instalar dependencia
pip install pyodbc

# Configurar credenciales en config.py
# → Editar SERVER, USERNAME, PASSWORD, DRIVER

# Preparar la BD en SQL Server (solo una vez)
# 1. sql/create_repositorio_sqlserver.sql
# 2. sql/create_tables_sqlserver.sql
# 3. sql/insert_tables_sqlserver.sql

# Ejecutar la aplicación
python streamucv_app.py
```

### 8.3 Resolución de Problemas Comunes

| Síntoma                          | Causa probable                               | Solución                                                                   |
|----------------------------------|----------------------------------------------|----------------------------------------------------------------------------|
| [IM002] Data source not found    | Driver ODBC no instalado o nombre incorrecto | Instalar ODBC Driver 17/18 for SQL Server; verificar nombre en config.py   |
| Login failed for user 'sa'       | Credenciales o modo de autenticación         | Habilitar autenticación SQL Server en SSMS; verificar usuario y contraseña |
| Cannot open database 'StreamUCV' | BD no creada o sin permisos                  | Ejecutar los 3 scripts en orden; GRANT CONNECT al usuario                  |

| Síntoma                       | Causa probable                    | Solución                                                  |
|-------------------------------|-----------------------------------|-----------------------------------------------------------|
| STRING_AGG WITHIN GROUP error | SQL Server < 2022                 | Usar STRING_AGG sin WITHIN GROUP en queries.py (Req 4)    |
| Treeview vacío sin error      | Esquema 'streaming' no encontrado | Verificar que create_repositorio se ejecutó correctamente |